

# **Advanced Sequential Files 1.**

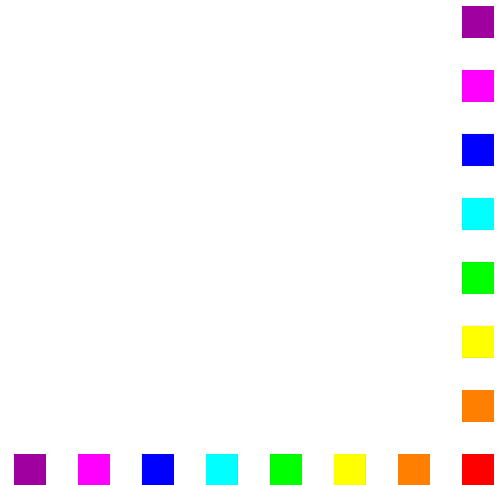
# Single Record Type Files

In a file which contains only one record type (the kind we have examined so far) the record structure is described as part of the file FD using an 01 level number.

The record description creates a 'buffer' capable of storing one record instance at a time.

Each time a record is read from the file it overwrites the previous contents of the buffer.

The record buffer is the only connection between the file and the program.



# Multiple Record Type Files

Quite often a single file will contain **more than one** type of record.

For instance, some of the terminal exercises required that your program apply a file of transaction records to the StudentsFile.

For simplicity, the Transaction file in these exercises contained one record type only; either Insertion or Update or Deletion.

In a real environment, transactions of this sort would normally be collected together into **one** single transaction file.



# Implications of a multiple record transaction file.

Gathering all the transactions into a single file implies that the file will contain **different record types** (i.e. records with different structures).

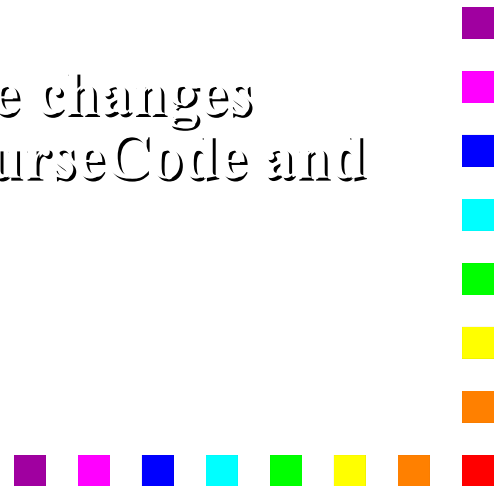
The different record structures may give rise to records which are also of **different lengths**.

For example

an insertion transaction will contain all the fields that appear in the StudentFile record (**32 characters**).

a deletion transaction will contain only the StudentId (**7 characters**).

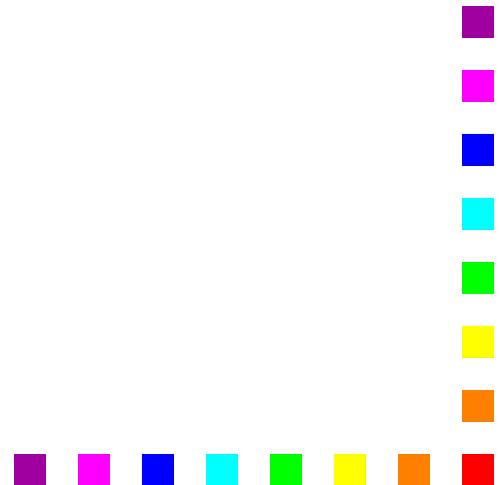
an update transaction used to record course changes might contain the StudentId, the OldCourseCode and the NewCourseCode (**15 characters**).



# Describing multiple record files

To describe these different record types we have to use **more than one** record description in the file's FD.

Because record descriptions always begin with level 01 we provide a 01 level for **each** record type in the file.



# Multiple record descriptions - One record buffer

```
DATA DIVISION.  
FILE SECTION.  
FD TransactionFile.  
01 InsertionRec.  
    02 StudentId          PIC 9(7) .  
    02 StudentName.  
        03 Surname        PIC X(8) .  
        03 Initials       PIC XX .  
    02 DateOfBirth.  
        03 YOBirth        PIC 9(2) .  
        03 MOBirth        PIC 9(2) .  
        03 DOBirth        PIC 9(2) .  
    02 CourseCode        PIC X(4) .  
    02 Grant              PIC 9(4) .  
    02 Gender             PIC X .  
  
01 DeleteRec.  
    02 StudentId          PIC 9(7) .  
  
01 UpdateRec.  
    02 StudentId          PIC 9(7) .  
    02 OldCourseCode      PIC X(4) .  
    02 NewCourseCode      PIC X(4) .
```

What is not obvious from this description is that COBOL continues to create just a **single 'record buffer'** for the file!

And this 'record buffer' is only able to store a **single record** at a time!

Multiple record descriptions in a file are **IMPLICITLY**  
**redefinitions** of the single record buffer.

### TransactionFile Buffer

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 9 | 2 | 3 | 0 | 1 | 6 | 5 | H | E | N | N | E | S | S | Y | R | M | 7 | 1 | 0 | 9 | 1 | 5 | L | M | 5 | 1 | 0 | 5 | 5 | 0 | F |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

Multiple record descriptions in a file are **IMPLICITLY redefinitions** of the single record buffer.

### TransactionFile Buffer

#### InsertionRec

| StudentId | StudentName | DateOfBirth | CourseCode | Grant | Gender |
|-----------|-------------|-------------|------------|-------|--------|
|-----------|-------------|-------------|------------|-------|--------|

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 9 | 2 | 3 | 0 | 1 | 6 | 5 | H | E | N | N | E | S | S | Y | R | M | 7 | 1 | 0 | 9 | 1 | 5 | L | M | 5 | 1 | 0 | 5 | 5 | 0 | F |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|



Multiple record descriptions in a file are **IMPLICITLY redefinitions** of the single record buffer.

## TransactionFile Buffer

### InsertionRec

| StudentId | StudentName | DateOfBirth | CourseCode | Grant | Gender |
|-----------|-------------|-------------|------------|-------|--------|
|-----------|-------------|-------------|------------|-------|--------|

9 2 3 0 1 6 5 H E N N E S S Y R M 7 1 0 9 1 5 L M 5 1 0 5 5 0 F

StudentId

**DeletionRec**

Multiple record descriptions in a file are **IMPLICITLY redefinitions** of the single record buffer.

## TransactionFile Buffer

### InsertionRec

| StudentId | StudentName | DateOfBirth | CourseCode | Grant | Gender |
|-----------|-------------|-------------|------------|-------|--------|
| 9230165   | HENNESSY    | RM710915    | LM510550   |       | F      |

StudentId  
**DeletionRec**

| StudentId | OldCourseCode | NewCourseCode |
|-----------|---------------|---------------|
|           |               |               |

**UpdateRec**

All these record descriptions are valid at the **same time**.  
But only one description makes sense for the values in  
the buffer.

How can we tell which description to use?

## TransactionFile Buffer

### InsertionRec

| StudentId | StudentName | DateOfBirth | CourseCode | Grant | Gender |
|-----------|-------------|-------------|------------|-------|--------|
| 9230165   | HENNESSY    | RM710915    | LM510550   |       | F      |

StudentId  
**DeletionRec**

| StudentId | OldCourseCode | NewCourseCode |
|-----------|---------------|---------------|
|           |               |               |

**UpdateRec**

# The Transaction Type Code

Generally we cannot reliably establish the type of record READ into the buffer by examining its contents.

To allow us to distinguish between the record types, a special data item is inserted into each transaction which identifies the transaction type.

This data item is usually the first data item in the transaction record and one character in size, but it does not have to be.

Transaction types can be identified using a number, a letter or other character.



# The Revised FD.

```
DATA DIVISION.
FILE SECTION.
FD TransactionFile.
01 InsertionRec.
    02 TransCode          PIC X.
    02 StudentId         PIC 9(7) .
    02 StudentName.
        03 Surname       PIC X(8) .
        03 Initials      PIC XX.
    02 DateOfBirth.
        03 YOBirth       PIC 9(2) .
        03 MOBirth       PIC 9(2) .
        03 DOBirth       PIC 9(2) .
    02 CourseCode        PIC X(4) .
    02 Grant             PIC 9(4) .
    02 Gender            PIC X.

01 DeleteRec.
    02 TransCode          PIC X.
    02 StudentId         PIC 9(7) .

01 UpdateRec.
    02 TransCode          PIC X.
    02 StudentId         PIC 9(7) .
    02 OldCourseCode     PIC X(4) .
    02 NewCourseCode     PIC X(4) .
```

TransCode occurs in all the record descriptions.

How can we refer to the one in DeleteRec?

**MOVE TransCode OF DeleteRec TO TCode.**

But TransCode really only needs to be defined in one record.

Since all the records map on to the **same area** of storage the TransCode defined for the InsertionRec can be used no matter which record type is actually in the buffer .

# The Final FD.

```
DATA DIVISION.
FILE SECTION.
FD TransactionFile.
01 InsertionRec.
   88 EndOfTransFile      VALUE HIGH-VALUES.
   02 TransCode            PIC X.
      88 Insertion        VALUE "I".
      88 Deletion         VALUE "D".
      88 Update           VALUE "U".
   02 StudentId            PIC 9(7).
   02 StudentName.
      03 Surname           PIC X(8).
      03 Initials         PIC XX.
   02 DateOfBirth.
      03 YOBirth           PIC 9(2).
      03 MOBirth           PIC 9(2).
      03 DOBirth           PIC 9(2).
   02 CourseCode           PIC X(4).
   02 Grant                PIC 9(4).
   02 Gender               PIC X.

01 DeleteRec.
   02 FILLER               PIC X(8).

01 UpdateRec.
   02 FILLER               PIC X(8).
   02 OldCourseCode        PIC X(4).
   02 NewCourseCode        PIC X(4).
```

TransCode and StudentId have the **same description** and are in the **same location** in all three records.

So they are defined only in the InsertionRec.

In the other records the area occupied by these two items is defined using **FILLER**.

What happens when we display the OldCourseCode?  
What happens if we now read an Update record into the buffer?

## TransactionFile Buffer

### InsertionRec

| TransCode | StudentId | StudentName | DateOfBirth | CourseCode | Grant | Gender |
|-----------|-----------|-------------|-------------|------------|-------|--------|
|-----------|-----------|-------------|-------------|------------|-------|--------|

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| I | 9 | 2 | 3 | 0 | 1 | 6 | 5 | H | E | N | N | E | S | S | Y | R | M | 7 | 1 | 0 | 9 | 1 | 5 | L | M | 5 | 1 | 0 | 5 | 5 | 0 | F |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

FILLER

DeletionRec

FILLER

| OldCourseCode | NewCourseCode |
|---------------|---------------|
|---------------|---------------|

UpdateRec

When a record smaller than the size of the largest record is read into the buffer any data that is not explicitly overwritten is left intact.

What happens when we display StudentName and DateOfBirth?

## TransactionFile Buffer

### InsertionRec

| TransCode | StudentId | StudentName | DateOfBirth | CourseCode | Grant | Gender |
|-----------|-----------|-------------|-------------|------------|-------|--------|
|-----------|-----------|-------------|-------------|------------|-------|--------|

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| U | 9 | 3 | 1 | 5 | 6 | 8 | 2 | L | M | 6 | 1 | L | M | 5 | 1 | R | M | 7 | 1 | 0 | 9 | 1 | 5 | L | M | 5 | 1 | 0 | 5 | 5 | 0 | F |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

FILLER

DeletionRec

FILLER

| OldCourseCode | NewCourseCode |
|---------------|---------------|
|---------------|---------------|

UpdateRec



# Printing a Report.

A program is required which will print a report.

The report, called the Student Details Report, will be based on the file Students.Dat.

The report will show the Name, StudentId, Gender and CourseCode of each student in the file.

A report is made up of groups of printed lines of different types.

What types of line are required for the Student Details Report?



# Report Print Lines.

Page Heading.

**UL Student Details Report**

Page Footing.

**Page : *PageNum***

Column Headings.

**Student Id. Student Name Gender Course**

Student Detail Line.

**StudentId. StudentName Gender CourseCode**

Report Footing.

**\*\*\* End of Student Details Report \*\*\***



# Describing Print Lines.

```
01 PageHeading.  
02 FILLER          PIC X(7) VALUE SPACES.  
02 FILLER          PIC X(25)  
    VALUE "UL Student Details Report".  
  
01 PageFootng.  
02 FILLER          PIC X(19) VALUE SPACES.  
02 FILLER          PIC X(7)  VALUE "Page : ".  
02 FILLER          PIC 99.  
  
01 ColumnHeadings PIC X(36)  
    VALUE " StudentId StudentName Gender Course".  
  
01 StudentDetailLine.  
02 PrnStudId      PIC BB9(7) .  
02 PrnStudName    PIC BBX(10) .  
02 PrnGender      PIC BBBBX.  
02 PrnCourse      PIC BBBBX(4) .  
  
01 ReportFootng   PIC X(38)  
    VALUE "*** End of Student Details Report ***".
```

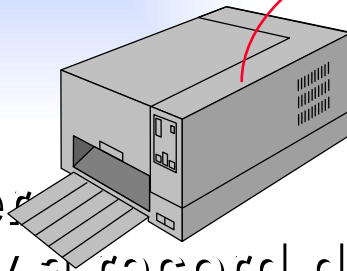
The Print Lines are all different **record** types!

# The File Buffer

All data coming from, or going to, the peripherals must pass through a file buffer declared in the File Section.

The file buffer is associated with the physical device by means of a Select and Assign clause.

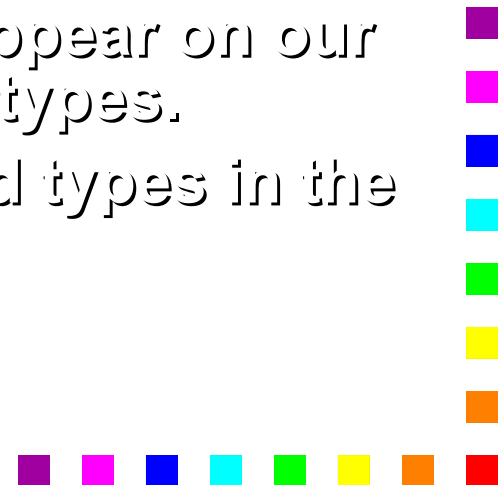
In previous lectures we saw that the file buffer is represented by a record description (or record).



```
ENVIRONMENT DIVISION.  
INPUT-OUTPUT SECTION.  
FILE-CONTROL.  
    SELECT Printer  
        ASSIGN TO "LPT1".  
  
DATA DIVISION.  
FILE SECTION.  
FD  Printer.  
01  PrintLine.  
    ??????????????????
```

But the different types of line that must appear on our report are declared as different record types.

How can we declare these different record types in the File Section?



# No VALUE clause in the FILE SECTION.

Defining a file buffer which is used by different record types is easy (as we have seen).

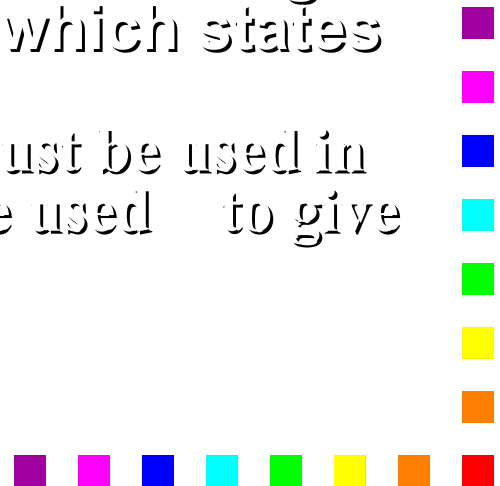
**But !!**

These record types all map on to the same area of storage and print line records **cannot** share the same area of storage.

Why? Because most of the print line record values are assigned using the VALUE clause and these values are assigned as soon as the program starts.

To prevent us trying to use the VALUE clause to assign values to a File buffer COBOL has a rule which states that;

In the FILE SECTION, the VALUE clause must be used in condition-name entries only (i.e. it cannot be used to give an initial value to an item).



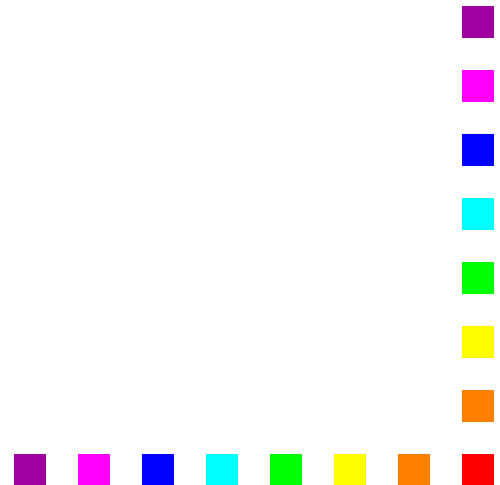
# A Solution

**We get round the problem as follows;**

**We define the print records in the WORKING-STORAGE SECTION.**

**We create a file buffer in the FILE SECTION which is the size of the largest print record.**

**We print a line by moving the appropriate print record to the file buffer and then WRITing the contents of the file buffer to the device.**



ENVIRONMENT DIVISION.  
INPUT-OUTPUT SECTION.  
FILE-CONTROL.

SELECT ReportFile ASSIGN TO "STUDENTS.RPT".

DATA DIVISION.

FILE SECTION.

FD ReportFile.

01 PrintLine PIC X(38).

WORKING-STORAGE SECTION.

01 PageHeading.

02 FILLER PIC X(7) VALUE SPACES.

02 FILLER PIC X(25)

VALUE "UL Student Details Report".

01 PageFooting.

02 FILLER PIC X(19) VALUE SPACES.

02 FILLER PIC X(7) VALUE "Page : ".

02 FILLER PIC 99.

01 ColumnHeadings PIC X(36)

VALUE " StudentId StudentName Gender Course".

01 StudentDetailLine.

02 PrnStudId PIC BB9(7).

02 PrnStudName PIC BBX(10).

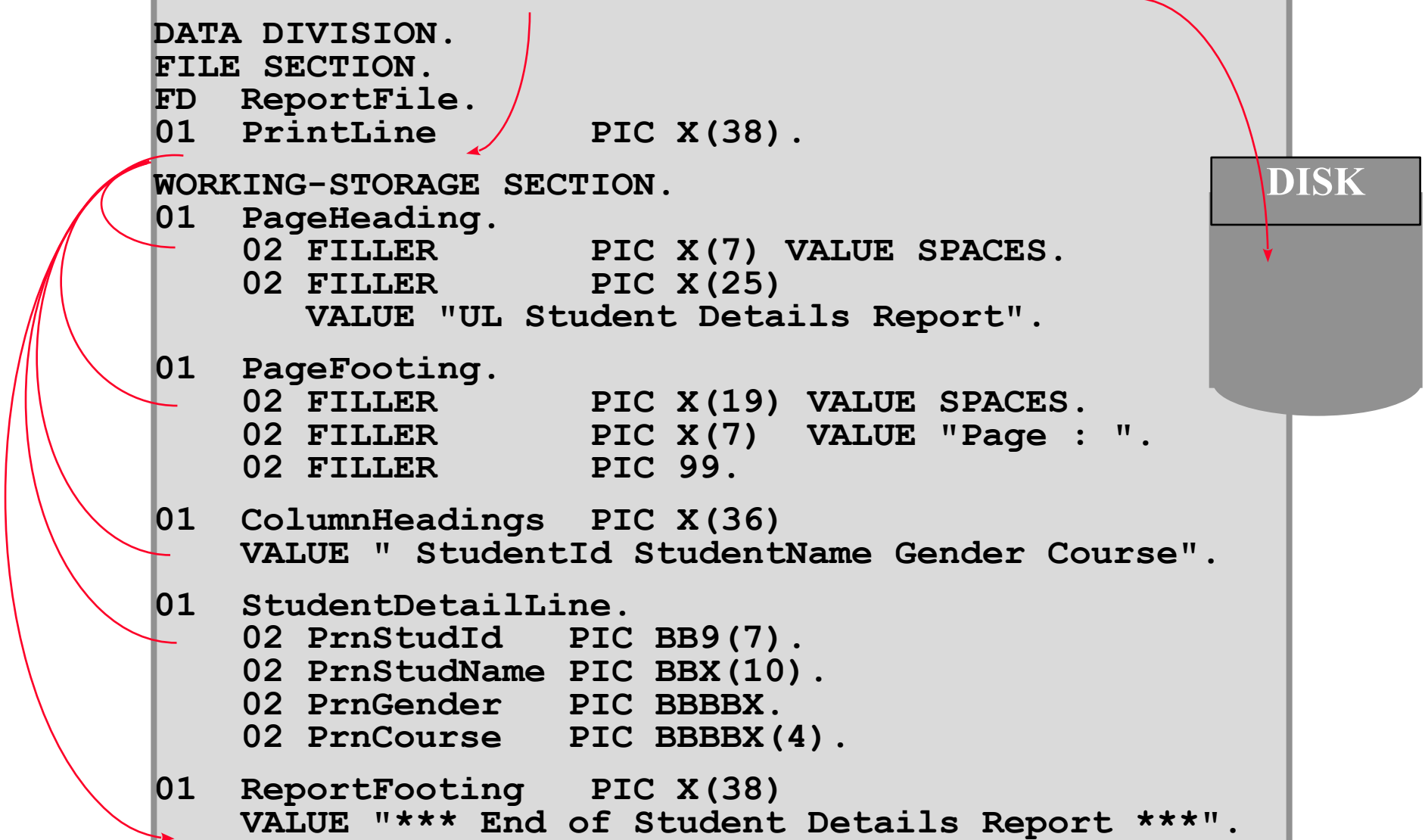
02 PrnGender PIC BBBBX.

02 PrnCourse PIC BBBBX(4).

01 ReportFooting PIC X(38)

VALUE "\*\*\* End of Student Details Report \*\*\*".

DISK



# WRITE Syntax revisited.

When we are writing to a **printer** or a **print file** we use a form of the WRITE command different from that we use when writing to a sequential file.

|   |   |               |   |           |   |              |   |   |   |              |
|---|---|---------------|---|-----------|---|--------------|---|---|---|--------------|
| [ | { | <u>BEFORE</u> | } | ADVANCING | { | AdvanceNum   | } | ] | ] |              |
|   |   |               |   |           |   |              |   |   |   | <u>LINE</u>  |
|   |   |               |   |           |   | <u>AFTER</u> |   |   |   | <u>LINES</u> |
|   |   |               |   |           |   |              |   |   |   |              |
|   |   |               |   |           |   | MnemonicName |   |   |   |              |
|   |   |               |   |           |   | <u>PAGE</u>  |   |   |   |              |





```
ENVIRONMENT DIVISION.  
INPUT-OUTPUT SECTION.  
FILE-CONTROL.  
    SELECT ReportFile ASSIGN TO "STUDENTS.RPT"  
        ORGANIZATION IS LINE SEQUENTIAL.
```

```
DATA DIVISION.  
FILE SECTION.  
FD  ReportFile.  
01  PrintLine          PIC X(40).
```

```
WORKING-STORAGE SECTION.  
01  HeadingLine        PIC X(21) VALUE "  Record Count Report".  
01  StudentTotalLine.  
    02 FILLER          PIC X(17) VALUE "Total Students = ".  
    02 PrnStudentCount PIC Z,ZZ9.  
01  MaleTotalLine.  
    02 FILLER          PIC X(17) VALUE "Total Males      = ".  
    02 PrnMaleCount    PIC Z,ZZ9.  
01  FemaleTotalLine.  
    02 FILLER          PIC X(17) VALUE "Total Females   = ".  
    02 PrnFemaleCount  PIC Z,ZZ9.
```

DISK

```
MOVE StudentCount TO PrnStudentCount  
MOVE MaleCount     TO PrnMaleCount  
MOVE FemaleCount   TO PrnFemaleCount  
  
WRITE PrintLine FROM HeadingLine      AFTER ADVANCING PAGE  
WRITE PrintLine FROM StudentTotalLine AFTER ADVANCING 2 LINES  
WRITE PrintLine FROM MaleTotalLine    AFTER ADVANCING 2 LINES  
WRITE PrintLine FROM FemaleTotalLine  AFTER ADVANCING 2 LINES.
```